TasteCast Testing, Sample Dataset, and Run-Based Artifact Spec for Cursor

Purpose

This document is a detailed implementation brief for a **practical, file-based testing workflow** for TasteCast before introducing any database.

The goal is to make the backend easy to test, easy to debug, and easy to **sample** validate end to **CSV fixtures**, **versioned run folders**, and **automated pytest coverage**.

This document is intentionally detailed so it can be given directly to Cursor/Copilot as a build spec.

---

1) What we are trying to solve

Right now, the simplest and most realistic way to test TasteCast is **not** to introduce a database first.

Instead, we should:

1. Create a **small set of sample datasets**.
2. Make each ingest/pipeline execution generate a **run_id**.
3. Persist canonical tables and output artifacts under a **run-based folder structure**.
4. Verify the pipeline manually on a few known cases.
5. Then add **automated tests** using pytest.

This gives us a clean local development and debugging loop.

It also matches our current architecture better because the backend already works by: - ingestir running a pipeline, - writing CSV artifacts, - exposing them through Flask API endpoints.

So the goal here is to formalize that workflow into something testable and reproducible.

---

2) High-level testing strategy

The testing plan should happen in this order:

Phase 1: Build one minimal end-to-end happy path

We need a known-good sample dataset that can run through the entire pipeline and generate outputs.

This means one fixture set with enough information to support: - sales history - menu items ingredient metadata - inventory snapshot - config values

Once that exists, we can manually test: - upload - pipeline execution - artifact generation - endpoint reads

Phase 2: Build controlled scenario datasets

After the happy path works, we need additional tiny datasets where we **already know what should happen**.

These should include: - a reorder-trigger case - a no-reorder case - a specials-trigger case - failure case - optionally a sparse/intermittent demand case

These datasets should be intentionally small and interpretable.

Phase 3: Add automated tests

Once sample fixtures and run-based artifacts exist, write pytest tests that: - run the ingest flow, are created, - verify API responses, - verify formulas and pipeline consistency, - verify fallback b error handling.

---

3) What Cursor should implement first

Cursor should implement the following in order:

1. **Fixture-based sample data files** under `tests/fixtures/`
2. **Run-based artifact persistence** under `artifacts/runs/<run_id>/`
3. **Manual end-to-end execution path**
4. **Basic integration tests** using pytest
5. **Controlled scenario tests**
6. **Consistency checks across artifacts and API responses**

Do **not** add a database yet. Do **not** add SQLite or Postgres in this step. Keep everything file-based a versioned by `run_id`.

---

4) Required run-based artifact design

Every time the backend ingests a dataset and runs the pipeline, it should create a unique `run_id`.

4.1 Run ID requirements

A `run_id` should be: - unique - stable for a single execution - readable enough for debugging in file paths

Recommended format:

`YYYYMMDDTHHMMSSZ_<short_random_suffix>`

Example:

`20260306T063012Z_ab12cd`

4.2 Artifact folder structure

Each run should be written under:

```
artifacts/
  runs/
    <run_id>/
      canonical/
      outputs/
      meta/
```

Example:

```
artifacts/
  runs/
    20260306T063012Z_ab12cd/
      canonical/
        sales_fact.csv
        menu_dim.csv
        recipe_fact.csv
        ingredient_dim.csv
        inventory_snapshot.csv
      outputs/
        daily_plan.csv
        ingredient_plan.csv
        advisories.csv
```

```
    forecast_summary.json
  meta/
    status.json
    validation_report.json
    config_used.yaml
    run_manifest.json
```

4.3 Why this structure matters

This structure makes it possible to: - inspect one run in isolation, - compare two runs, - de keep reproducible artifacts, - test without a DB, - serve a specific run through API queries.

---

5) API behavior changes needed for testing

The API should support both “latest run” behavior and explicit `run_id` selection.

5.1 Ingest response

`POST /api/ingest` should return something like:

```
{
  "status": "success",
  "run_id": "20260306T063012Z_ab12cd",
  "message": "Pipeline completed successfully",
  "artifacts_created": [
    "daily_plan.csv",
    "ingredient_plan.csv",
    "advisories.csv",
    "forecast_summary.json"
  ]
}
```

If fallback occurs:

```
{
  "status": "fallback",
  "run_id": "20260306T063012Z_ab12cd",
  "message": "Primary model failed; fallback pipeline used",
  "warnings": ["recipe map missing", "using forecast-only mode"]
}
```

5.2 Read endpoints

These endpoints should optionally accept `run_id` as a query param:

- `GET /api/daily-plan?run_id=<run_id>`
- `GET /api/ingredient-plan?run_id=<run_id>`
- `GET /api/advisories?run_id=<run_id>`
- `GET /api/forecast?run_id=<run_id>`

If no `run_id` is provided, they may default to the most recent successful run.

5.3 Helpful debug endpoints

For local development, add helper endpoints if needed:

- `GET /api/runs`
- `GET /api/run/<run_id>/status`
- `GET /api/run/<run_id>/files`
- `GET /api/run/<run_id>/manifest`

These are optional but extremely useful for debugging.

---

6) Sample dataset strategy

We should not rely on one giant sample CSV.

Instead, we should define **multiple small fixture sets**, each designed to test a specific behavior.

6.1 Minimum fixture families

We need at least these fixture sets:

1. `happy_path/`
2. `reorder_case/`
3. `no_reorder_case/`
4. `specials_case/`
5. `bad_schema_case/`
6. `sparse_case/` (optional but strongly recommended)

Each fixture set should live under:

```
tests/fixtures/
  happy_path/
```

```
reorder_case/
no_reorder_case/
specials_case/
bad_schema_case/
sparse_case/
```

6.2 Files inside each fixture family

For full-pipeline cases, each fixture family should usually contain:

- `sales.csv`
- `menu.csv`
- `recipes.csv`
- `ingredients.csv`
- `inventory.csv`
- `config.yaml`

If a case is meant to fail validation, it may intentionally omit one or more files or columns.

---

7) Detailed sample dataset specification

Below is the exact structure Cursor should create.

---

8) Happy path fixture

Purpose

This is the first dataset we use to prove the system runs end to end.

It should: - be small, - be realistic, - include 2–3 menu items, - include enough history forecasting, - include recipes and inventory, - generate non-empty daily plan and advisories outputs.

Recommended folder

```
tests/fixtures/happy_path/
```

Required files

- `sales.csv`
- `menu.csv`

- `recipes.csv`
- `ingredients.csv`
- `inventory.csv`
- `config.yaml`

Recommended menu setup

Use one store and three items: - pizza - salad - soup

These are simple enough to understand but rich enough to test recipe expansion.

Example `sales.csv`

```
date,store_id,menu_item_id,qty_sold,price,promo_flag
2026-02-01,store_1,pizza,20,12.99,0
2026-02-01,store_1,salad,8,9.99,0
2026-02-01,store_1,soup,6,6.99,0
2026-02-02,store_1,pizza,22,12.99,0
2026-02-02,store_1,salad,7,9.99,0
2026-02-02,store_1,soup,5,6.99,0
2026-02-03,store_1,pizza,30,12.99,1
2026-02-03,store_1,salad,10,9.99,0
2026-02-03,store_1,soup,7,6.99,0
2026-02-04,store_1,pizza,24,12.99,0
2026-02-04,store_1,salad,9,9.99,0
2026-02-04,store_1,soup,6,6.99,0
2026-02-05,store_1,pizza,21,12.99,0
2026-02-05,store_1,salad,8,9.99,0
2026-02-05,store_1,soup,6,6.99,0
2026-02-06,store_1,pizza,26,12.99,0
2026-02-06,store_1,salad,11,9.99,0
2026-02-06,store_1,soup,8,6.99,0
2026-02-07,store_1,pizza,32,12.99,1
2026-02-07,store_1,salad,12,9.99,0
2026-02-07,store_1,soup,9,6.99,0
```

This does not need to be huge. Even `7–21` days is fine for a first fixture if the fallback forecast logic can still run.

Example `menu.csv`

```
menu_item_id,menu_item_name,category
pizza,Cheese Pizza,entree
```

```
salad,House Salad,side
soup,Tomato Soup,starter
```

Example `recipes.csv`

```
menu_item_id,ingredient_id,ingredient_qty_per_unit
pizza,dough,1.0
pizza,cheese,0.5
pizza,sauce,0.2
salad,lettuce,0.3
salad,dressing,0.1
soup,broth,0.4
soup,cream,0.1
```

Example `ingredients.csv`

```
ingredient_id,ingredient_name,lead_time_days_avg,min_order_qty,lot_size,case_multiple,shelf_life_
DUMMY_IGNORE,Dummy,1,1,1,1,1,0.95
```

Do **not** actually use the dummy row above. Use real rows like this:

```
ingredient_id,ingredient_name,lead_time_days_avg,min_order_qty,lot_size,case_multiple,shelf_life_
dough,Dough,2,20,20,10,3,0.95
cheese,Cheese,2,10,10,5,7,0.95
sauce,Sauce,3,10,10,5,10,0.95
lettuce,Lettuce,1,10,10,5,4,0.95
dressing,Dressing,3,5,5,5,14,0.90
broth,Broth,2,10,10,5,7,0.95
cream,Cream,1,10,10,5,5,0.95
```

Example `inventory.csv`

```
store_id,ingredient_id,snapshot_date,on_hand_qty,on_order_qty,backorder_qty,usable_qty
store_1,dough,2026-03-01,40,0,0,40
store_1,cheese,2026-03-01,30,0,0,30
store_1,sauce,2026-03-01,25,0,0,25
store_1,lettuce,2026-03-01,20,0,0,20
store_1,dressing,2026-03-01,20,0,0,20
store_1,broth,2026-03-01,20,0,0,20
store_1,cream,2026-03-01,15,0,0,15
```

Example `config.yaml`

```
forecast:
  horizon_days: 7
  default_model: fallback
inventory:
  default_service_level: 0.95
  use_safety_stock: true
specials:
  enabled: true
  allowed_weekdays: [4, 5, 6]
  max_special_units_per_day: 10
```

Expected behavior

This fixture should: - ingest successfully, - generate a run_id, - write canonical files, - write c return non-empty daily plan, - return at least zero or more advisories, - return a usable forecast summary.

---

9) Reorder case fixture

Purpose

This dataset should specifically trigger a **BUY** advisory for one or more ingredients.

The simplest way to do this is: - keep forecasted demand moderately high, - set on-hand inv optionally set lead time to more than `1` day, - optionally add safety stock in config.

Folder

```
tests/fixtures/reorder_case/
```

Main idea

Pizza demand should imply dough demand that exceeds available inventory over the lead time window.

Example `sales.csv`

```
date,store_id,menu_item_id,qty_sold,price,promo_flag
2026-02-01,store_1,pizza,25,12.99,0
```

```
2026-02-02,store_1,pizza,24,12.99,0
2026-02-03,store_1,pizza,28,12.99,0
2026-02-04,store_1,pizza,26,12.99,0
2026-02-05,store_1,pizza,29,12.99,0
2026-02-06,store_1,pizza,30,12.99,0
2026-02-07,store_1,pizza,31,12.99,0
```

Example `menu.csv`

```
menu_item_id,menu_item_name,category
pizza,Cheese Pizza,entree
```

Example `recipes.csv`

```
menu_item_id,ingredient_id,ingredient_qty_per_unit
pizza,dough,1.0
pizza,cheese,0.5
```

Example `ingredients.csv`

```
ingredient_id,ingredient_name,lead_time_days_avg,min_order_qty,lot_size,case_multiple,shelf_life_
dough,Dough,2,20,20,10,3,0.95
cheese,Cheese,2,10,10,5,7,0.95
```

Example `inventory.csv`

```
store_id,ingredient_id,snapshot_date,on_hand_qty,on_order_qty,backorder_qty,usable_qty
store_1,dough,2026-03-01,10,0,0,10
store_1,cheese,2026-03-01,30,0,0,30
```

Example `config.yaml`

```
forecast:
  horizon_days: 3
inventory:
  default_service_level: 0.95
  use_safety_stock: true
  default_safety_stock_units: 10
```

Expected behavior

This case should produce at least one advisory like: - `BUY` - `ingredient = dough` - `qty > 0`

Depending on the implementation, it may also produce a message explaining: - lead-time dema reorder point, - inventory shortfall, - rounded order quantity.

What should be asserted in tests

At minimum: `advisories.csv` contains a dough buy `suggested_order_qty_final > 0` - `ingredient_plan.csv` shows dough inventory pressure

---

1 0) No-reorder case fixture

Purpose

This is the counterpart to the reorder case.

It should use similar sales history but with enough on-hand inventory that no dough buy is needed.

Folder

```
tests/fixtures/no_reorder_case/
```

Example

You can reuse the same sales/menu/recipes/ingredients as `reorder_case`, but increase inventory.

Example `inventory.csv`

```
store_id,ingredient_id,snapshot_date,on_hand_qty,on_order_qty,backorder_qty,usable_qty
store_1,dough,2026-03-01,100,0,0,100
store_1,cheese,2026-03-01,80,0,0,80
```

Expected behavior

This case should **not** generate a `BUY` advisory for dough.

What should be asserted

- no dough BUY advisory exists
- daily plan still exists
- forecast still exists
- ingredient plan still exists

---

11) Specials case fixture

Purpose

This fixture should trigger a specials recommendation because there is projected surplus inventory of going unused or expiring.

Folder

```
tests/fixtures/specials_case/
```

Main idea

Set up: - relatively low baseline demand, - high on-hand inventory of a short-shelf-life ingredient, item that consumes that ingredient, - specials enabled on allowed weekdays.

Example use case

Suppose lettuce inventory is very high, salad demand is modest, and lettuce has short shelf lif system should recommend a salad special.

Example `sales.csv`

```
date,store_id,menu_item_id,qty_sold,price,promo_flag
2026-02-01,store_1,salad,5,9.99,0
2026-02-02,store_1,salad,4,9.99,0
2026-02-03,store_1,salad,6,9.99,0
2026-02-04,store_1,salad,5,9.99,0
2026-02-05,store_1,salad,5,9.99,0
2026-02-06,store_1,salad,4,9.99,0
2026-02-07,store_1,salad,5,9.99,0
```

Example `menu.csv`

```
menu_item_id,menu_item_name,category
salad,House Salad,side
```

Example `recipes.csv`

```
menu_item_id,ingredient_id,ingredient_qty_per_unit
salad,lettuce,0.4
salad,dressing,0.1
```

Example `ingredients.csv`

```
ingredient_id,ingredient_name,lead_time_days_avg,min_order_qty,lot_size,case_multiple,shelf_life_
lettuce,Lettuce,1,10,10,5,3,0.95
dressing,Dressing,3,5,5,5,14,0.90
```

Example `inventory.csv`

```
store_id,ingredient_id,snapshot_date,on_hand_qty,on_order_qty,backorder_qty,usable_qty
store_1,lettuce,2026-03-01,100,0,0,100
store_1,dressing,2026-03-01,20,0,0,20
```

Example `config.yaml`

```
forecast:
  horizon_days: 5
specials:
  enabled: true
  allowed_weekdays: [4, 5, 6]
  max_special_units_per_day: 10
  surplus_threshold_units: 15
```

Expected behavior

This case should produce some advisory indicating a special or surplus `SPECIAL` such as: - `SPECIAL_LETTUCE` - `EXCESS_WASTE_RISK`

The implementation can choose naming, but the behavior should clearly reflect surplus logic.

Important consistency check

If a special is ac`special_added > 0` `qty_total = baseline_qty + special_added` - ingredient demand must reflect `qty_total`, not baseline only

This is one of the most important tests in the entire system.

---

1 2) Bad schema case fixture

Purpose

This fixture ensures validation works properly.

Folder

```
tests/fixtures/bad_schema_case/
```

Variants

Create at least one invalid sales file that is missing required columns.

Example invalid `sales.csv`

```
date,store_id,menu_item_id,price,promo_flag
2026-02-01,store_1,pizza,12.99,0
2026-02-02,store_1,pizza,12.99,0
```

This file is missing `qty_sold`.

Expected behavior

Ingest should fail validation and return a clear error.

Possible response:

```
{
  "status": "error",
```

```
  "message": "Missing required column: qty_sold"
}
```

What should be asserted

- API response is an error or `4xx`
- no successful output artifacts are written
- validation report is written if applicable

---

13) Sparse case fixture

Purpose

This tests low-volume or intermittent demand.

Folder

```
tests/fixtures/sparse_case/
```

Main idea

Use a menu item with many zero-demand days.

Example `sales.csv`

```
date,store_id,menu_item_id,qty_sold,price,promo_flag
2026-02-01,store_1,cake,0,5.99,0
2026-02-02,store_1,cake,0,5.99,0
2026-02-03,store_1,cake,2,5.99,0
2026-02-04,store_1,cake,0,5.99,0
2026-02-05,store_1,cake,0,5.99,0
2026-02-06,store_1,cake,1,5.99,0
2026-02-07,store_1,cake,0,5.99,0
```

Expected behavior

If sparse routing exists, this item should either: - use fallback logic, - or at least not crash the model.

What should be asserted

- ingest succeeds,
- forecast exists,
- output is nonnegative,
- fallback or sparse-mode status is visible if implemented.

---

14) Canonical tables that should be written for each run

Even when input files use varied column names, the pipeline should normalize them and write tables.

At minimum, we want:

14.1 `sales_fact.csv`

Columns: - `date` - `store_id` - `menu_item_id` - `qty_sold` - optional other columns

14.2 `menu_dim.csv`

Columns: - `menu_item_id` - `menu_item_name` - `category`

14.3 `recipe_fact.csv`

Columns: - `menu_item_id` - `ingredient_id` - `ingredient_qty_per_unit`

14.4 `ingredient_dim.csv`

Columns: - `ingredient_id` `ingredient_name` `lead_time_days_avg` `min_order_qty` - `lot_size` - `case_multiple` - `shelf_life_days` - `service_level_target`

14.5 `inventory_snapshot.csv`

Columns: `store_id` `ingredient_id` `snapshot_date` `on_hand_qty` `on_order_qty` - `backorder_qty` - `usable_qty`

Writing these canonical files makes debugging much easier and proves the mapping layer works.

---

15) Outputs that must be written per run

At minimum, each successful run should write:

15.1 `daily_plan.csv`

Should include at le`date` `store_id` `menu_item_id` `baseline_qty` `special_added` - `qty_total`

Optional but recommended: - `forecast_p10` - `forecast_p50` - `forecast_p90` - `prep_target`

15.2 `ingredient_plan.csv`

Should include at least: `date` - `store_id` - `ingredient_id` - `ingredient_demand` - `begin_on_hand` - `end_on_hand` - `inventory_position` - `suggested_order_qty_raw` - `suggested_order_qty_final`

15.3 `advisories.csv`

Should include at least: - `date` - `type` - `ingredient` or `entity_id` - `qty` - `message`

15.4 `forecast_summary.json`

Should include at least: - `points` - `total_forecast` - `avg_daily`

15.5 `status.json`

Should include: - `run_id` - `status` - `started_at` - `completed_at` - `warnings` - `fallback_used`

15.6 `validation_report.json`

Should capture: - missing columns - type coercions - duplicate rows - warnings

15.7 `run_manifest.json`

Should list all generated files and paths.

---

16) Manual test workflow

Before writing a lot of automated tests, the backend should support a clean manual test flow.

Cursor should create either: - a shell script, - a Python script, - or a README section

that shows exactly how to run a fixture end to end.

16.1 Example manual test flow

Step 1

Start the backend locally.

Step 2

Upload a fixture set using the happy path fixture.

Step 3

Capture the returned `run_id`.

Step 4

Inspect the filesystem:

```
artifacts/runs/<run_id>/
```

Step 5

Open the files: `canonical/sales_fact.csv` - `outputs/daily_plan.csv` - `outputs/advisories.csv` - `outputs/forecast_summary.json` - `meta/status.json`

Step 6

Call the API endpoints: - `/api/daily-plan?run_id=<run_id>` - `/api/advisories?run_id=<run_id>` - `/api/forecast?run_id=<run_id>`

Step 7

Confirm the responses match the written artifacts.

---

17) Automated test plan

Cursor should then add pytest coverage.

17.1 Test folder structure

```
tests/
  fixtures/
    happy_path/
    reorder_case/
    no_reorder_case/
    specials_case/
    bad_schema_case/
    sparse_case/
  expected/
  test_ingest.py
  test_artifacts.py
  test_api_endpoints.py
  test_reorder_logic.py
  test_specials_logic.py
  test_validation.py
  test_consistency.py
```

---

18) Test categories in detail

18.1 Ingest tests

These verify that fixture input can be uploaded and processed.

Test: happy path ingest succeeds

Assert: - HTTP success response - `run_id` exists - status is `success` or `fallback`

Test: bad schema ingest fails

Assert: - HTTP error or `4xx` - clear validation message

18.2 Artifact creation tests

These verify the run folder and output files are created.

Test: run directory exists

Assert: - `artifacts/runs/<run_id>/` exists

Test: canonical files exist

Assert existence of: - `sales_fact.csv` - `menu_dim.csv` - `recipe_fact.csv` - `ingredient_dim.csv` - `inventory_snapshot.csv`

Test: output files exist

Assert existence of: `daily_plan.csv` - `ingredient_plan.csv` - `advisories.csv` - `forecast_summary.json`

Test: meta files exist

Assert existence of: - `status.json` - `validation_report.json` - `run_manifest.json`

18.3 API endpoint tests

Test: `/api/daily-plan`

Assert: - returns `200` - response not empty for happy path - response matches CSV row c least key totals

Test: `/api/advisories`

Assert: - returns `200` - response shape is correct

Test: `/api/forecast`

Assert: - returns `200` `total_forecast` equals or approximately equals sum of plan quantities depending on the intended contract

---

18.4 Reorder logic tests

Test: reorder case triggers dough BUY

Use `reorder_case` fixture.

Assert: - advisory exists with type BUY - ingredient/entity is dough - quantity > `0`

Test: no-reorder case does not trigger dough BUY

Use `no_reorder_case` fixture.

Assert: - no dough BUY advisory exists

18.5 Specials logic tests

Test: specials case creates surplus-related advisory

Use `specials_case` fixture.

Assert: - advisory exists with type SPECIAL c`special_added > 0` for at least one daily plan row

Test: specials consistency

Assert for each affected row: - `qty_total == baseline_qty + special_added`

Also assert that ingredient demand increased appropriately after specials were applied.

18.6 Validation tests

Test: missing required column is caught

Use `bad_schema_case`.

Assert: - error message mentions the missing column - no successful outputs are produced

---

18.7 Consistency tests

These are especially important.

Test: forecast summary matches daily plan

If `forecast_summary.total_forecast` is intended to represent total baseline forecast, verify it equa the correct sum. If it is intended to represent total planned production, verify it equals `sum(qty_total)`.

The contract must be explicit.

Test: ingredient demand matches recipe expansion

For known rows, verify:

`ingredient_demand = qty_total * ingredient_qty_per_unit`

or the more complete yield-adjusted version if implemented.

Test: advisory quantities align with ingredient plan

If ingredient plan says suggested order for dough is `20`, advisories should not say `300` u
procurement rules intentionally round to `300`. If it rounds, the reason should explain that.

Test: clear-data behavior

If clear-data endpoint exists, verify the backend returns empty state after clear and does not
repopulate demo data.

---

19) Suggested expected-value strategy

For tiny fixtures, Cursor should create tests that verify **exact values** where possible.

This is especially useful for: - reorder quantity math, - recipe expansion, - specials arithmetic,
totals.

Example exact-value check

If: - forecasted dough demand over lead time = `50` - safety stock = `10` - inventory pos
`15`

Then:

```
raw_order = 50 + 10 - 15 = 45
```

If case multiple is `10`:

```
final_order = 50
```

A test should assert the final order quantity is exactly `50`.

---

20) Formula checks that tests should verify

At minimum, tests should cover these formulas when implemented.

20.1 Quantity total

```
qty_total = baseline_qty + special_added
```

20.2 Ingredient demand

```
ingredient_demand = Σ(qty_total * recipe_qty_per_unit)
```

20.3 Inventory position

```
inventory_position = usable_on_hand + on_order - backorder_qty
```

20.4 Reorder point

```
reorder_point = lead_time_demand + safety_stock
```

20.5 Raw order quantity

```
raw_order_qty = max(0, target_stock - inventory_position)
```

20.6 Rounded order quantity

Rounded to pack size / lot size / case multiple as configured.

Tests do not need to cover every formula immediately, but these are the highest priority.

---

21) What Cursor should do to make local debugging easy

Cursor should add developer-friendly utilities.

21.1 Run manifest

For every run, w`run_manifest.json` containing: - run_id - fixture name if known - list of gen
files - timestamps

21.2 Logging

Add readable logs for: - ingest start/end - validation warnings - pipeline start/end - artifact wr
fallback mode activation

21.3 Optional inspection script

Create a small script like:

```
python scripts/show_run.py <run_id>
```

that prints: - status - files - summary stats

This is optional but very useful.

---

2 2) What not to overcomplicate yet

Cursor should **not** do these things in this phase:

- add SQLite
- add Postgres
- build a full migrations layer
- overengineer model training
- create huge synthetic datasets
- add dozens of fixture families immediately
- add cloud object storage yet

The objective is a **small, deterministic, inspectable** file-based testing setup.

---

2 3) Recommended first implementation milestones

Milestone 1

Create `happy_path` fixture and make it run end to end.

Milestone 2

Add run-based artifact folders and `run_id` support.

Milestone 3

Expose `run_id` through API read endpoints.

Milestone 4

Add `reorder_case` and `no_reorder_case` fixtures.

Milestone 5

Add pytest integration tests for ingest + artifact existence + endpoint reads.

Milestone 6

Add `specials_case` and consistency checks.

Milestone 7

Add validation failure fixture and tests.

---

2 4) Exact implementation request for Cursor

Below is the direct instruction block Cursor should follow.

Cursor Implementation Request

I want to set up a practical file-based testing workflow for TasteCast before adding any database.

Please implement the following in order:

1 . Add fixture-based sample datasets

Create the following folders ur `tests/fixtures/`: - `happy_path/` - `reorder_case/` - `no_reorder_case/` `specials_case/` `bad_schema_case/` `sparse_case/` (optional if time is limited, but preferred)

For full-pipeline fixtures, include these fil `sales.csv` - `menu.csv` - `recipes.csv` - `ingredients.csv` - `inventory.csv` - `config.yaml`

Make the datasets tiny, realistic, and easy to reason about.

2 . Add run-based artifact persistence

Every pipeline run should generate a unique `run_id` and write to:

```
artifacts/runs/<run_id>/
  canonical/
```

```
outputs/
meta/
```

Write at least these files when available:

Canonical: - `sales_fact.csv` `menu_dim.csv` `recipe_fact.csv` `ingredient_dim.csv` - `inventory_snapshot.csv`

Outputs: - `daily_plan.csv` - `ingredient_plan.csv` - `advisories.csv` - `forecast_summary.json`

Meta: - `status.json` - `validation_report.json` - `config_used.yaml` - `run_manifest.json`

3. Update the API

Make `POST /api/ingest` return a `run_id`.

Allow these endpoints to optionally `run_id`: `/api/daily-plan` `/api/ingredient-plan` - `/api/advisories` - `/api/forecast`

If no `run_id` is provided, it can default to the most recent successful run.

4. Add a manual local test path

Create a simple way to run the happy path locally and inspect outputs. This can be a READ script, or Python script.

It should clearly document how to: - start the backend - upload the `run_id` fixture - get t inspect generated files - call the API endpoints for that run

5. Add pytest tests

Create tests that verify:

Ingest

- happy path ingest succeeds
- bad schema ingest fails cleanly

Artifact creation

- run folder exists
- canonical files exist
- output files exist
- meta files exist

API responses

- `/api/daily-plan` returns data for happy path
- `/api/advisories` returns data for happy path
- `/api/forecast` returns valid summary data

Reorder logic

- `reorder_case` produces a BUY advisory for dough (or the chosen target ingredient)
- `no_reorder_case` does not produce that BUY advisory

Specials logic

- `specials_case` produces a specials/surplus advisory
- `qty_total == baseline_qty + special_added`
- ingredient demand reflects `qty_total`

Consistency

- forecast summary totals match the intended totals from the plan
- advisory quantities align with ingredient plan logic

6. Keep the system simple

Do not add any database yet. Do not overengineer the architecture. Keep it file-based, determ
inspectable, and easy to test.

7. If some current code paths do not support all fixture files yet

Adapt the pipeline minimally so the fixture-driven tests can run. Prefer small refactors over a full rewrite.

---

25) Final goal

After this work is done, we should be able to:

1. Run a sample fixture end to end.
2. Receive a `run_id`.
3. Inspect versioned artifacts on disk.
4. Query those artifacts through the API.
5. Prove with automated tests that:
6. ingest works,
7. artifacts exist,
8. endpoints work,
9. reorder logic works,
10. specials logic is internally consistent,

11. validation errors are handled correctly.

That is the cleanest and lowest-friction testing system for TasteCast right now.

It avoids database complexity while still giving us reproducible runs, debuggable outputs, and s
confidence in the pipeline.